Lecture 11 - Processor Scheduling
Crash Study Guide

Operating Systems - CS x61 | Dr. Noha Adly

Exam survival focus

Master the difference between long-, medium-, and short-term scheduling, then learn how each scheduling algorithm chooses the next process. For numerical questions, always draw the Gantt chart first, then calculate finish time, turnaround time, waiting time, and normalized turnaround time.

0. Emergency study order

If your exam is very soon, study in this order. Do not try to memorize everything equally.

1. Definitions and scheduling levels: scheduler, dispatcher, long-term, medium-term, short-term.
2. State transitions and when short-term scheduling is invoked.
3. Preemptive vs non-preemptive scheduling.
4. Metrics and formulas: turnaround time, waiting time, response time, normalized turnaround time, response ratio.
5. Algorithms: FCFS, RR, Virtual RR, SPN, SRT, HRRN, Feedback / Multilevel Feedback Queue.
6. Special topics: fair-share scheduling, thread scheduling ULT vs KLT, traditional UNIX and Solaris scheduling.
7. Practice Gantt chart questions. These are very likely in exams.

One-line memory hook

Long-term admits processes. Medium-term swaps/suspends processes. Short-term/dispatcher chooses the next ready process to run on the CPU.

1. What processor scheduling means

The CPU provides execution time. Many processes compete for that time. Scheduling is the OS decision about which ready process gets the CPU next.

- Scheduler: OS module that makes scheduling decisions for CPU, I/O devices, and other resources.
- CPU scheduling is needed whenever the CPU is idle and there are ready processes waiting.
- Ready processes are stored in a ready queue or multiple ready queues.
- Scheduling can happen because of I/O request, I/O completion, process interruption, or process termination.

2. Types of processor scheduling

Type	Main question	Where it acts	Key exam point
Long-term scheduling	Should a new process be admitted?	New -> Ready or New -> Ready/Suspend	Controls the degree of multiprogramming. Too many processes means each gets less CPU. Too few means CPU may be idle.
Medium-term scheduling	Should a suspended process be brought into memory?	Ready/Suspend -> Ready and Blocked/Suspend -> Blocked	Part of memory management. It performs swapping and controls system load.
Short-term scheduling	Which ready process runs next?	Ready -> Running	Executes very frequently. Also called the dispatcher.

Important diagram logic: in the two-suspend-state process model, long-term scheduling admits new jobs, medium-term scheduling activates or suspends processes, and short-term scheduling dispatches a ready process to running.

3. When short-term scheduling is invoked

- Absolutely required when a process exits or becomes blocked, because the CPU must choose another ready process.
- May be required when a new process is created, when I/O completes, or when a clock interrupt occurs.
- A clock interrupt matters mainly in preemptive scheduling because it may force the running process back to the ready queue.

State-transition mini-map

new -> ready -> running -> terminated
running -> blocked when waiting for I/O/event
blocked -> ready when event occurs
running -> ready on interrupt/yield/preemption

4. Preemptive vs non-preemptive scheduling

Concept	Meaning	When CPU changes owner	Pros / cons
Non-preemptive	Once a process gets the CPU, it runs until it voluntarily gives it up.	Process terminates or blocks for I/O/service.	Less context-switch overhead, but one process can monopolize the CPU.
Preemptive	The OS can interrupt a running process and move it back to Ready.	New higher-priority process arrives, I/O completion makes a process ready, or clock interrupt/time slice expires.	Better response and fairness, but more overhead due to context switching.

Most modern operating systems use preemptive CPU scheduling implemented through timer interrupts.

5. Different system needs

System type	Main goal	Scheduling implication
Batch systems	Finish jobs and maximize throughput.	Preemptive or non-preemptive may be acceptable. Turnaround time matters.
Interactive systems	Fast response to users.	Preemption is important so one process cannot monopolize the CPU.
Real-time systems	Meet deadlines.	Processes must run and finish on time; priorities/deadlines matter more than average waiting time.

6. CPU bursts, I/O bursts, and process behavior

- Processes alternate between CPU bursts and I/O waits.
- CPU-bound processes have long CPU bursts and use the processor heavily.
- I/O-bound processes have short CPU bursts and frequently block waiting for I/O.
- Most processes have many short CPU bursts, which is why scheduling should not be designed only for long CPU-bound jobs.
- As CPUs get faster, many processes appear more I/O-bound because they finish computation quickly and then wait for I/O.

7. Response time and turnaround time

Metric	Meaning	Best for
Response time	Time between submitting a request and receiving the first output.	Interactive users.
Turnaround time (TAT)	Total time a job spends in the system from arrival/submission to completion.	Batch jobs.
Waiting time	Total time a process waits in the ready queue.	Gantt chart calculations.
Normalized turnaround time	Turnaround time divided by service time.	Comparing short and long jobs fairly.
Throughput	Completed jobs per unit time.	System-oriented performance.
Processor utilization	Percentage of time CPU is busy.	System efficiency.

Response-time thresholds from the lecture

Less than 0.1 second: response to key presses/mouse clicks.
Less than 1 second: keeps user attention for thought-intensive activities.
1-2 seconds: important for terminal activities and remembering information across responses.
2-4 seconds: inhibits concentration.
4-15 seconds: user may lose track of the operation.
15+ seconds: usually rules out conversational interaction.

8. Formulas you must know

Formula	Meaning	How to use it
Turnaround time = Finish time - Arrival time	Total residence time in the system.	Use after drawing the Gantt chart.
Waiting time = Turnaround time - Service time	How long process waited without running.	Equivalent to start-arrival only for simple non-preemptive cases.
Normalized turnaround time = Turnaround time / Service time	Relative delay.	High value means the job suffered compared with its actual CPU need.
Response ratio R = (W + S) / S = 1 + W/S	HRRN priority score.	Choose the ready process with the highest R. W = waiting time, S = service time.
S_next = alpha*T_last + (1-alpha)*S_last	Exponential averaging prediction.	Used to estimate future CPU burst/service time. Larger alpha reacts faster to recent behavior.

9. Context switching is expensive

A process switch is not free. The OS must enter kernel mode, save the old process state, choose another process, possibly reset memory-management data, load the new process state, and return to user mode.

- Direct cost: actual time spent performing the switch.
- Indirect cost: cache invalidation/reload, memory effects, and possibly page-related costs.
- Too many process switches can consume CPU time instead of doing useful work.
- This is why Round Robin quantum length is a major design decision.

10. Algorithm 1 - First Come First Served (FCFS)

- Processes are executed in order of arrival.
- Usually non-preemptive in the lecture examples.
- Oldest process in the ready queue runs next.
- Easy to implement with a simple queue.
- Fair in arrival order, but bad for short jobs stuck behind long jobs.
- Convoy effect: many short jobs wait behind one long CPU-bound job.
- Favors CPU-bound processes; I/O-bound processes may wait even though they would quickly block and release CPU.

Classic FCFS trap

P1=24, P2=3, P3=3 arriving at time 0. If order is P1, P2, P3, average waiting time = (0+24+27)/3 = 17. If order is P2, P3, P1, average waiting time = (0+3+6)/3 = 3. Same jobs, different arrival order, huge difference.

11. Algorithm 2 - Round Robin (RR)

- Preemptive version of FCFS using a time quantum/time slice.
- Each process gets a small unit of CPU time, commonly 20-50 ms.
- When the clock interrupt occurs, the running process is moved to the end of the ready queue if it is not finished.
- Good for interactive systems because no process can monopolize the CPU.
- Favors processor-bound processes unless adjusted, because I/O-bound processes often give up CPU early and then return to the end of the queue.

Quantum choice	Effect
Too short	Short processes move quickly, but too many context switches reduce CPU efficiency. Example: q=4 ms and switch=1 ms -> overhead = 1/(4+1) = 20%.
Too long	Low overhead, but poor response for short interactive requests. Example: q=100 ms with 1 ms switch -> about 1% overhead.
Good guide	Quantum should be slightly larger than the time required for a typical interaction, so interactive jobs block before their quantum expires.

12. Virtual Round Robin (VRR)

- Designed to fix unfairness of RR toward I/O-bound processes.
- RR problem: I/O-bound processes give up the CPU early, then wait behind CPU-bound processes that used their whole quantum.
- VRR adds an auxiliary queue for processes released from I/O blocking.
- At dispatch time, auxiliary-queue processes get preference over the main ready queue.
- After they use their remaining/allowed time, they return to the normal queue.

13. Algorithm 3 - Shortest Process Next (SPN/SJF)

- Also called Shortest Job First.
- Non-preemptive.
- Selects the ready process with the shortest expected processing time.
- Short processes jump ahead of long processes, improving average response/turnaround.
- Needs an estimate of the service time, which is difficult.
- Can starve long processes if short processes keep arriving.
- Less predictable for long jobs.

14. Estimating execution time

SPN and SRT need an estimate of how long a process will run. In batch systems, the programmer may provide an estimate. In interactive systems, the OS can estimate future CPU bursts from past CPU bursts.

Method	Idea	Problem / advantage
Simple average	Use past observed CPU bursts equally.	Old behavior counts as much as recent behavior, which may be inaccurate.
Exponential averaging	S_next = alpha*T_last + (1-alpha)*S_last.	Recent observations can be given more weight.
Large alpha, e.g. 0.8	Responds quickly to recent changes.	Can be jerky if there is a temporary spike.
Small alpha, e.g. 0.2	Smooths over more past observations.	Responds slowly to real behavior changes.

15. Algorithm 4 - Shortest Remaining Time (SRT)

- Preemptive version of SPN.
- If a new process arrives with estimated processing time less than the remaining time of the currently running process, the running process is preempted.
- Usually gives better turnaround time than SPN because short jobs get immediate preference.
- Does not favor long jobs like FCFS does.
- Does not need periodic clock interrupts like RR, but the OS must track elapsed service time.
- Still risks starvation of long jobs.

16. Algorithm 5 - Highest Response Ratio Next (HRRN)

- Non-preemptive.
- Chooses the ready process with the highest response ratio R = (W+S)/S.
- Favors short jobs because smaller S gives a bigger ratio.
- Also prevents starvation better than SPN because W grows while a process waits.
- Long processes eventually become attractive because their waiting time increases.

HRRN mental example

Process A: W=10, S=2 -> R=(10+2)/2=6.
Process B: W=20, S=10 -> R=(20+10)/10=3.
Choose A because 6 > 3. If B waits longer, its ratio will rise.

17. Priority scheduling

- Each process has a priority.
- Scheduler chooses a ready process from the highest-priority non-empty queue.
- Multiple ready queues can represent different priority levels.
- Static priorities can cause starvation of lower-priority processes.
- Dynamic priorities can change based on age or execution history.
- The lecture warns: all scheduling policies can be viewed as priority scheduling, because every algorithm has some rule that gives one process preference over another.

18. Algorithm 6 - Feedback / Multilevel Feedback Queue

- Used when it is hard to predict remaining time, so SPN/SRT/HRRN are not practical.
- Main idea: favor short jobs by penalizing jobs that have already used lots of CPU.
- Preemptive and uses dynamic priority.
- A new process enters the highest-priority queue RQ0.
- Each time it is preempted and returns to ready state, it is demoted to a lower-priority queue.
- Within each queue, FCFS or RR can be used. At the lowest queue, a process usually stays there and is treated Round Robin.
- Short jobs finish quickly near the top. Long jobs gradually drift downward.
- Can starve long jobs if new short jobs keep entering.

MLFQ parameter	Meaning
Number of queues	How many priority levels exist.
Scheduling algorithm per queue	For example RR in high queues, FCFS in lowest queue.
Upgrade method	When a waiting process is promoted to avoid starvation.
Demotion method	When CPU-heavy processes are moved downward.
Entry rule	Which queue a new or returning process enters.

Example MLFQ from the lecture

Q0: RR with quantum 8 ms.
Q1: RR with quantum 16 ms.
Q2: FCFS.
A new job starts in Q0. If it does not finish in 8 ms, move to Q1. If it does not finish in another 16 ms, move to Q2.

19. Worked comparison from the lecture example

Processes: A(Arrival 0, Service 3), B(2,6), C(4,4), D(6,5), E(8,2). Service time means total required CPU execution time.

Algorithm	Mean turnaround time	Mean normalized turnaround	Exam interpretation
FCFS	8.60	2.56	Simple, but can make short later jobs suffer.
RR q=1	10.80	2.71	Responsive but more switching and overhead.
RR q=4	10.00	2.71	Less switching than q=1 but still not best here.
SPN	7.60	1.84	Good average, but non-preemptive and may starve long jobs.
SRT	7.20	1.59	Best in this example, but requires remaining-time estimates.
HRRN	8.00	2.14	Balances short-job preference with aging.
Feedback q=1	10.00	2.29	Dynamic queues; may be worse for some workloads.
Feedback q=2^(i-1)	10.60	2.63	Longer quanta in lower queues.

20. Algorithm summary table

Algorithm	Preemptive?	Selection rule	Strength	Weakness
FCFS	No	Oldest ready process first.	Simple and fair by arrival.	Convoy effect; bad for short/I/O-bound jobs.
RR	Yes	FCFS with time quantum.	Good interactive response.	Quantum choice is sensitive; context-switch overhead.
VRR	Yes	RR plus auxiliary queue for I/O-returned processes.	Improves I/O-bound response.	More complex than basic RR.
SPN/SJF	No	Shortest expected service time.	Good average turnaround.	Needs estimates; long job starvation.
SRT	Yes	Shortest remaining time.	Excellent turnaround for short jobs.	Needs estimates and tracking; long job starvation.
HRRN	No	Highest (W+S)/S.	Aging reduces starvation.	Needs service-time estimate; non-preemptive.
Priority	Can be either	Highest priority first.	Good for importance/deadlines.	Low priorities can starve.
Feedback/MLFQ	Yes	New jobs high priority; CPU users demoted.	Works without knowing service time.	Long jobs may suffer unless aging/promotions exist.

21. How to solve scheduling numerical questions

8. Write the process table: arrival time and service time for each process.
9. Choose the algorithm rule. Ask: preemptive or non-preemptive?
10. Draw a timeline/Gantt chart from time 0.
11. Whenever the CPU becomes free or a preemption event occurs, list the ready processes.
12. Apply the algorithm rule to choose the next process.
13. Record each process finish time.
14. Compute turnaround time = finish - arrival.
15. Compute waiting time = turnaround - service.
16. Compute normalized turnaround = turnaround / service.

17. Average the requested metric.

Common calculation traps

Do not use start-arrival as waiting time for preemptive algorithms. Use waiting = turnaround - service.
For RR, if a process blocks/finishes before quantum ends, immediately schedule the next ready process.
For SRT, re-check remaining times whenever a new process arrives.
For HRRN, recompute R only when choosing the next process.

22. Fair-Share Scheduling (FSS)

- Normal scheduling may be unfair when one user runs many processes and another user runs only one.
- FSS schedules based on groups of processes belonging to users or applications.
- Each user or group is assigned a share of the processor.
- The OS monitors usage: users/groups that already consumed more than their fair share get fewer resources; those below their fair share get more.
- FSS uses execution history of each process and execution history of the related group.
- Dynamic priority considers base priority, recent CPU usage of the process, and recent CPU usage of its group.
- Important trap: higher numerical priority value means lower actual priority in the lecture.

FSS example logic

If A is in group 1 and B,C are in group 2, and both groups have share 0.5, group 2 should not get twice the CPU simply because it has two processes. FSS tries to allocate fairly by group, not only by individual process.

23. Scheduling algorithm evaluation

Evaluation method	How it works	Advantages	Disadvantages
Deterministic modeling	Use a fixed workload and run it through different algorithms, then calculate statistics.	Simple, fast, exact for that workload.	Too specific; needs exact knowledge; tied to example data.
Queuing theory	Use probability distributions for CPU and I/O bursts and model the system as servers with queues.	More general and mathematical.	Requires statistical assumptions and can be complex.

Lecture example for deterministic modeling: with burst times P1=10, P2=29, P3=3, P4=7, P5=12, the shown average waiting times are FCFS=28, SPN=13, and RR with quantum 10=23. The point is not that one result is universal; the point is that the answer depends on workload.

24. Thread scheduling

When processes have multiple threads, scheduling can happen at process level or thread level depending on whether threads are user-level or kernel-level.

Feature	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Who schedules?	Kernel schedules the process. A thread library inside the process schedules its threads.	Kernel schedules individual threads directly.
Kernel awareness	Kernel is unaware of user threads.	Kernel knows about the threads.
Preemption of threads	No timer interrupt between user threads inside the process unless the thread library gets control.	Kernel can preempt threads with a quantum.
Blocking behavior	If one ULT blocks in a blocking system call, the whole process may be blocked depending on implementation.	If one KLT blocks, kernel can schedule another thread.
Switch cost	Thread switch is cheap, often a handful of machine instructions.	Fuller context switch, more expensive.
Scheduling intelligence	Application can use application-specific scheduling.	Kernel can balance threads across all processes.

ULT vs KLT exam answer

ULT is faster to switch and can use application-specific choices, but the kernel cannot directly schedule its threads. KLT is more expensive but gives the kernel full visibility and better global scheduling/control.

25. Traditional UNIX scheduling

- Designed for good response time for interactive users while avoiding starvation of low-priority background jobs.
- Uses multilevel feedback / fair-share style scheduling with Round Robin inside each priority queue.
- Uses 1-second preemption.
- Priority is based on process type and execution history.
- Priorities are recomputed once per second.
- Priority bands, in decreasing priority: swapper, block I/O device control, file manipulation, character I/O device control, user processes.
- nice allows a user to voluntarily reduce their process priority.
- Execution history penalizes CPU-bound processes and favors I/O-bound/interactive behavior.

26. Solaris scheduling

- Solaris schedules kernel threads.
- It has six scheduling classes.
- Default class is time-sharing based on a multilevel feedback queue.
- For the exam, connect Solaris with KLT and multilevel feedback/time-sharing scheduling.

27. Professor-style traps and what to answer

Question / trap	Correct answer
Is short-term scheduler the same as dispatcher?	In this lecture, yes: short-term scheduling chooses which ready process runs next and is known as the dispatcher.
Does long-term scheduling run frequently?	No. Short-term runs most frequently. Long-term controls admission and degree of multiprogramming.
Can a non-preemptive process be interrupted by the scheduler?	No, it gives up CPU voluntarily by terminating or blocking.
Why is RR quantum important?	Too short -> too much context switching. Too long -> bad interactive response.
Why can FCFS be bad?	Convoy effect: short/I/O-bound jobs wait behind long CPU-bound jobs.
Why can SPN/SRT starve long jobs?	A continuous supply of short jobs can keep long jobs waiting.
How does HRRN reduce starvation?	Waiting time W increases the response ratio, so old jobs eventually get high priority.
Why use feedback scheduling?	It avoids needing service-time prediction by judging processes based on how much CPU they already used.
What is normalized turnaround time?	TAT divided by service time; it measures relative delay.
Why is switching expensive?	Save/load process state, kernel transition, MMU/cache effects, and indirect memory performance hits.

28. Likely exam questions with compact answers

Q: Define processor scheduling.

A: It is the OS activity of deciding which ready process/thread receives CPU execution time.

Q: Compare long-, medium-, and short-term scheduling.

A: Long-term admits new processes and controls degree of multiprogramming. Medium-term swaps/suspends and activates processes. Short-term dispatches one ready process to run next.

Q: When is short-term scheduling definitely needed?

A: When the running process exits or becomes blocked.

Q: Define preemptive scheduling.

A: The OS can interrupt a running process and move it back to Ready, usually due to timer interrupt, new process arrival, or I/O completion.

Q: What is the convoy effect?

A: Many short jobs wait behind one long job, especially in FCFS.

Q: What is the main difference between FCFS and RR?

A: RR is FCFS with preemption/time slicing.

Q: Why can RR perform badly with a very small quantum?

A: Too many context switches create high overhead and reduce CPU efficiency.

Q: What does SPN need that FCFS does not?

A: An estimate or knowledge of service/execution time.

Q: What is SRT?

A: The preemptive version of SPN; it runs the process with the shortest remaining time.

Q: What is HRRN formula?

A: R = (W + S) / S, where W is waiting time and S is service time.

Q: How does feedback scheduling work?

A: New jobs start high priority; jobs that keep using CPU are demoted to lower-priority queues.

Q: What is FSS?

A: Fair-share scheduling allocates CPU based on users/groups, not only individual processes, using process and group execution history.

Q: Compare ULT and KLT scheduling.

A: ULT is scheduled inside the process with kernel unaware of threads; KLT is scheduled by the kernel as individual threads.

Q: What is deterministic modeling?

A: Evaluate algorithms by running a fixed workload through them and calculating metrics.

29. Flashcards

Term	Meaning
Scheduler	OS module that decides resource allocation, especially CPU execution time.
Dispatcher	Short-term scheduler that picks next ready process to run.
Degree of multiprogramming	Number of active processes in the system; controlled by long-term scheduling.
Ready queue	Queue of processes ready to run but waiting for CPU.
Preemption	Interrupting a running process and returning it to Ready.
CPU-bound	Long CPU bursts, heavy CPU use.
I/O-bound	Short CPU bursts, frequent I/O waits.
Turnaround time	Finish time minus arrival time.
Normalized turnaround	Turnaround time divided by service time.
Convoy effect	Short jobs stuck behind long job.
Time quantum	Maximum CPU time in one RR turn.
SPN	Non-preemptive shortest expected service time first.
SRT	Preemptive shortest remaining time first.
HRRN	Highest response ratio next, R=(W+S)/S.
Feedback scheduling	Dynamic priority queues that demote CPU-heavy jobs.
Fair-share scheduling	CPU allocation by user/group share and usage history.

30. Final one-page cheat sheet

The whole lecture in one page

Scheduling = choosing who gets CPU time.
Long-term = admit jobs, controls multiprogramming. Medium-term = swapping/suspend/activate. Short-term = dispatcher, Ready -> Running.
Non-preemptive = process runs until exit/block. Preemptive = OS can interrupt and resume later.
Metrics: TAT=Finish-Arrival. Waiting=TAT-Service. Normalized=TAT/Service. HRRN R=(W+S)/S.
FCFS: simple, fair, convoy effect, bad for short/I/O-bound.

RR: time quantum, good response; too short = overhead, too long = poor interactivity.
VRR: auxiliary queue for processes returning from I/O.
SPN: shortest expected job, non-preemptive, good average, starvation risk.
SRT: preemptive SPN, best short-job response, starvation risk.
HRRN: aging + short-job preference; long jobs eventually rise.
Feedback/MLFQ: starts high, demotes CPU-heavy jobs, can promote to prevent starvation.
FSS: fairness by users/groups, not just individual processes. Higher numeric dynamic priority means lower actual priority.
ULT: kernel schedules process; app schedules threads. KLT: kernel schedules threads.
Traditional UNIX: multilevel feedback/fair-share, RR queues, 1-second preemption, priority bands, nice, penalizes CPU-bound.